

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
SIMATS ENGINEERING
COMPUTER SCIENCE AND ENGINEERING
CSA17 - ARTIFICIAL INTELLIGENCE

Experiment No. 1: Python Program to Solve the 8-Puzzle Problem

Aim

To write a Python program to solve the 8-Puzzle problem, which involves rearranging tiles on a 3x3 board from a given initial state to a desired goal state using the minimum number of moves, by applying a state-space search technique (A* Search / Best-First Search).

Algorithm

1. Start.
2. Represent the 8-puzzle board as a 3x3 list, with 0 denoting the blank tile.
3. Read the initial state and the goal state of the puzzle.
4. Define a function to locate the position (row, col) of the blank tile (0) in the current state.
5. Define a function to generate all valid successor states by sliding the blank tile Up, Down, Left, or Right, discarding moves that go outside the 3x3 grid.
6. Define a heuristic function $h(n)$ that calculates the number of misplaced tiles (or Manhattan distance) between the current state and the goal state.
7. Implement the search (A* / Best-First Search) using a priority queue (min-heap), where each node stores the state, the path cost $g(n)$, and $f(n) = g(n) + h(n)$.
8. Push the initial state into the priority queue and mark it as visited.
9. While the priority queue is not empty: a) Pop the state with the lowest $f(n)$ value. b) If this state equals the goal state, stop and print the solution path. c) Otherwise, generate its successors, compute $f(n)$ for each, and push unvisited successors into the queue.
10. If the queue becomes empty without reaching the goal state, report that no solution exists.
11. Print the sequence of moves / intermediate states from the initial state to the goal state.
12. Stop.

Pseudocode

```
BEGIN
  INPUT: initial_state, goal_state
  CREATE priority_queue PQ
  INSERT (initial_state, g=0, f=h(initial_state)) into PQ
  CREATE empty set VISITED

  WHILE PQ is not empty DO
    current = PQ.pop_min_f()

    IF current.state == goal_state THEN
      PRINT "Solution found"
      PRINT path(current)
      BREAK
    END IF
```

```

    ADD current.state to VISITED

    FOR each successor in generate_moves(current.state) DO
        IF successor NOT IN VISITED THEN
            g_new = current.g + 1
            h_new = heuristic(successor, goal_state)
            f_new = g_new + h_new
            INSERT (successor, g_new, f_new) into PQ
        END IF
    END FOR
END WHILE

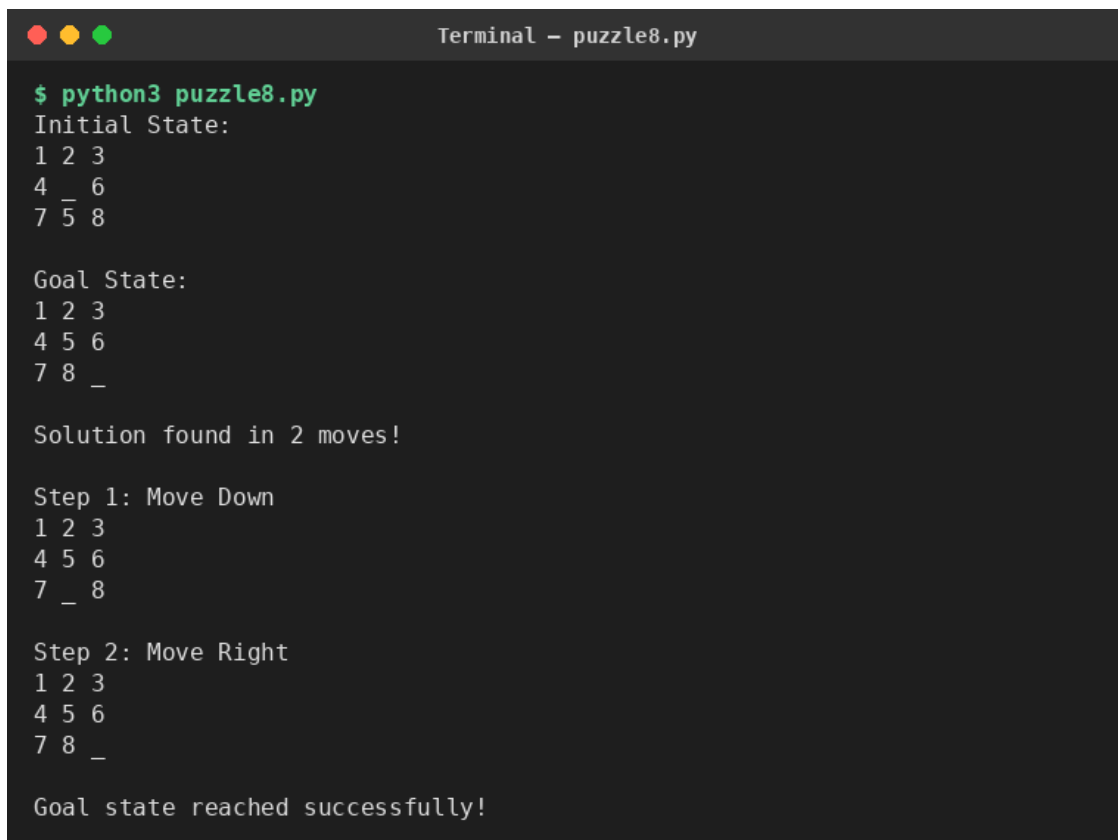
IF PQ is empty AND goal not reached THEN
    PRINT "No solution exists"
END IF
END

```

Result

The Python program for the 8-Puzzle Problem was executed successfully. The program accepted the initial state and the goal state of the puzzle as input, searched through the possible states using the search algorithm, and produced the sequence of moves required to transform the initial state into the goal state. The goal state was reached, and the minimum number of moves along with the intermediate states were displayed as output. Thus, the 8-Puzzle problem was solved successfully using Python.

Output Screenshot:



```

Terminal - puzzle8.py

$ python3 puzzle8.py
Initial State:
1 2 3
4 _ 6
7 5 8

Goal State:
1 2 3
4 5 6
7 8 _

Solution found in 2 moves!

Step 1: Move Down
1 2 3
4 5 6
7 _ 8

Step 2: Move Right
1 2 3
4 5 6
7 8 _

Goal state reached successfully!

```